

# NỀN TẢNG MÁY TÍNH

Phần Cứng — Hệ Điều Hành — Mạng

Hướng dẫn toàn diện cho kỹ sư phần mềm  
Khởi đầu từ những nguyên lý cơ bản nhất

Biên soạn bởi: DigitalCore Team

Phiên bản: V1.0

Ngày phát hành: 2026-08-06

# MỤC LỤC

---

## PHẦN I: PHẦN CỨNG MÁY TÍNH

- 1.1. Tổng quan về kiến trúc máy tính
- 1.2. Bộ xử lý trung tâm (CPU)
- 1.3. Bộ nhớ (Memory)
- 1.4. Thiết bị lưu trữ (Storage)
- 1.5. Thiết bị nhập/xuất (I/O)
- 1.6. Bus và giao tiếp phần cứng

## PHẦN II: HỆ ĐIỀU HÀNH

- 2.1. Giới thiệu hệ điều hành
- 2.2. Kernel – Nhân hệ điều hành
- 2.3. Quản lý tiến trình (Process Management)
- 2.4. Quản lý bộ nhớ (Memory Management)
- 2.5. Quản lý tập tin (File System)
- 2.6. Quản lý thiết bị (Device Management)
- 2.7. Hệ điều hành phổ biến

## PHẦN III: MẠNG MÁY TÍNH

- 3.1. Giới thiệu mạng máy tính
- 3.2. Mô hình OSI và TCP/IP
- 3.3. Các giao thức mạng cơ bản
- 3.4. Địa chỉ IP và định tuyến
- 3.5. Các thiết bị mạng
- 3.6. Bảo mật mạng cơ bản

## PHẦN IV: CÁCH MÁY TÍNH THỰC SỰ HOẠT ĐỘNG

- 4.1. Từ code đến thực thi
- 4.2. Chu trình boot và khởi động
- 4.3. Hệ thống nhúng và hệ điều hành
- 4.4. Tối ưu hóa và hiệu năng

## PHẦN V: TỔNG KẾT VÀ ỨNG DỤNG

- 5.1. Tích hợp các thành phần
- 5.2. Best practices cho kỹ sư
- 5.3. Tài nguyên học tập

# PHẦN I: PHẦN CỨNG MÁY TÍNH

## 1.1. Tổng quan về kiến trúc máy tính

Phần cứng máy tính là tập hợp tất cả các thành phần vật lý cấu thành nên một hệ thống máy tính. Hiểu biết về phần cứng là nền tảng quan trọng để trở thành một kỹ sư phần mềm giỏi, vì nó giúp bạn tối ưu hóa code, debug hiệu quả, và đưa ra các quyết định kiến trúc đúng đắn.

Một hệ thống máy tính hiện đại bao gồm các thành phần chính sau:

| STT | Thiết bị     | Mô tả                      | Chức năng chính                                       |
|-----|--------------|----------------------------|-------------------------------------------------------|
| 1   | CPU          | Bộ xử lý trung tâm         | Thực thi các lệnh và xử lý dữ liệu                    |
| 2   | RAM          | Bộ nhớ truy cập ngẫu nhiên | Lưu trữ dữ liệu tạm thời cho các tiến trình đang chạy |
| 3   | Ổ cứng/SSD   | Thiết bị lưu trữ           | Lưu trữ dữ liệu lâu dài                               |
| 4   | Motherboard  | Bo mạch chủ                | Kết nối tất cả các thành phần phần cứng               |
| 5   | GPU          | Bộ xử lý đồ họa            | Xử lý đồ họa và tính toán song song                   |
| 6   | PSU          | Bộ nguồn                   | Cung cấp điện cho các thành phần                      |
| 7   | Thiết bị I/O | Nhập/xuất                  | Giao tiếp với người dùng và thiết bị ngoại vi         |

## 1.2. Bộ xử lý trung tâm (CPU)

CPU (Central Processing Unit) là “bộ não” của máy tính, chịu trách nhiệm thực thi các lệnh và xử lý dữ liệu. CPU hiện đại là một trong những thành phần phức tạp nhất trong hệ thống máy tính.

### Cấu trúc cơ bản của CPU:

- ALU (Arithmetic Logic Unit): Đơn vị thực hiện các phép toán số học và logic (cộng, trừ, nhân, chia, AND, OR, NOT, v.v.)
- CU (Control Unit): Đơn vị điều khiển, giải mã các lệnh và điều phối hoạt động của các thành phần khác
- Registers: Các thanh ghi tốc độ cao để lưu trữ dữ liệu tạm thời (ACC, PC, IR, SP, v.v.)
- Cache: Bộ nhớ đệm tốc độ cao (L1, L2, L3) để giảm thời gian truy cập dữ liệu

### Chu trình lệnh cơ bản (Fetch-Decode-Execute):

1. Fetch: CPU lấy lệnh từ bộ nhớ (RAM) vào Instruction Register (IR)
2. Decode: CU giải mã lệnh để xác định loại thao tác cần thực hiện
3. Execute: ALU thực hiện thao tác được yêu cầu
4. Store: Kết quả được lưu trữ lại bộ nhớ hoặc register

Quá trình này lặp lại hàng tỷ lần mỗi giây. CPU hiện đại có thể thực thi hàng tỷ lệnh mỗi giây (GHz = tỷ chu kỳ mỗi giây).

### CPU hiện đại – Các tính năng tiên tiến:

- Multi-core: Nhiều nhân xử lý độc lập trên cùng một chip (dual-core, quad-core, octa-core, v.v.)

- Hyper-threading / SMT: Mỗi nhân vật lý có thể xử lý nhiều luồng logic đồng thời
- Pipelining: Các giai đoạn của cycle lệnh được thực hiện song song cho nhiều lệnh khác nhau
- Branch Prediction: Dự đoán nhánh để tối ưu hóa việc thực thi lệnh
- Out-of-Order Execution: Thực thi các lệnh theo thứ tự tối ưu thay vì thứ tự trong chương trình
- Vector Processing (SIMD): Xử lý nhiều dữ liệu cùng lúc với cùng một lệnh (SSE, AVX, NEON)

### 1.3. Bộ nhớ (Memory)

Bộ nhớ máy tính được tổ chức theo một hệ thống phân cấp (memory hierarchy) để cân bằng giữa tốc độ, dung lượng và chi phí.

#### Hệ thống phân cấp bộ nhớ:

| Cấp độ | Loại bộ nhớ         | Tốc độ     | Dung lượng điển hình | Chi phí/GB |
|--------|---------------------|------------|----------------------|------------|
| L1     | Registers trong CPU | Nhanh nhất | Vài KB               | Rất cao    |
| L2     | Cache L1/L2         | Rất nhanh  | Vài KB – MB          | Cao        |
| L3     | Cache L3            | Nhanh      | Vài MB – vài chục MB | Trung bình |
| 4      | RAM (Main Memory)   | Trung bình | 4GB – 128GB+         | Thấp       |
| 5      | SSD/HDD (Storage)   | Chậm       | 256GB – nhiều TB     | Rất thấp   |

#### RAM (Random Access Memory):

- DRAM (Dynamic RAM): Loại RAM phổ biến nhất, cần được refresh liên tục để giữ dữ liệu
- SRAM (Static RAM): Nhanh hơn, đắt hơn, dùng cho cache
- DDR (Double Data Rate): Công nghệ RAM hiện đại (DDR4, DDR5) truyền dữ liệu ở cả hai cạnh của clock signal
- Virtual Memory: Kỹ thuật sử dụng một phần ổ cứng như RAM mở rộng

#### Bộ nhớ chỉ đọc (ROM):

- BIOS/UEFI: Chứa firmware khởi động máy
- Flash Memory: Dùng trong SSD, USB, thẻ nhớ

### 1.4. Thiết bị lưu trữ (Storage)

Thiết bị lưu trữ được sử dụng để lưu giữ dữ liệu lâu dài, ngay cả khi mất điện.

#### Ổ cứng HDD (Hard Disk Drive):

- Sử dụng đĩa từ quay và đầu đọc/ghi
- Dung lượng lớn, chi phí thấp
- Tốc độ chậm hơn SSD (50–200 MB/s)
- Có bộ phận cơ khí, dễ hỏng do va đập

#### Ổ cứng SSD (Solid State Drive):

- Sử dụng chip NAND flash
- Tốc độ cao hơn nhiều (200 MB/s – 7+ GB/s cho NVMe)

- Không có bộ phận chuyển động, bền hơn
- Chi phí cao hơn HDD
- NVMe: Giao thức hiện đại, sử dụng PCIe bus, tốc độ cực cao

### Cấu trúc file system cơ bản:

- Block/Sector: Đơn vị lưu trữ nhỏ nhất (thường 512 bytes hoặc 4KB)
- File Allocation Table: Theo dõi vị trí các file trên disk
- Inode (Linux/Unix): Chứa metadata của file
- Journaling: Ghi log các thay đổi để phục hồi khi crash

## 1.5. Thiết bị nhập/xuất (I/O)

Thiết bị I/O cho phép máy tính giao tiếp với thế giới bên ngoài – người dùng và các thiết bị khác.

### Thiết bị nhập (Input):

- Bàn phím, chuột, touchpad • Microphone, camera • Scanner, cảm biến • Network interface (nhận dữ liệu)

### Thiết bị xuất (Output):

- Màn hình (monitor, display) • Loa, tai nghe • Máy in • Network interface (gửi dữ liệu)

### Controller và Driver:

- I/O Controller: Chip chuyên biệt quản lý giao tiếp với thiết bị I/O
- Device Driver: Phần mềm cho phép hệ điều hành giao tiếp với phần cứng
- Interrupts: Tín hiệu từ thiết bị I/O yêu cầu CPU chú ý
- DMA (Direct Memory Access): Cho phép thiết bị I/O truy cập RAM trực tiếp mà không cần CPU

## 1.6. Bus và giao tiếp phần cứng

Bus là hệ thống “đường cao tốc” cho phép các thành phần phần cứng giao tiếp với nhau.

### Các loại bus chính:

- Data Bus: Truyền dữ liệu giữa các thành phần
- Address Bus: Xác định địa chỉ bộ nhớ hoặc thiết bị
- Control Bus: Truyền tín hiệu điều khiển

### Các chuẩn bus hiện đại:

- PCIe (PCI Express): Bus tốc độ cao cho GPU, SSD NVMe, card mạng — PCIe 3.0 1 GB/s per lane; PCIe 4.0 2 GB/s; PCIe 5.0 4 GB/s
- USB: USB 2.0: 480 Mbps; USB 3.0/3.1: 5–10 Gbps; USB 3.2/4: 20–40 Gbps
- SATA: 6 Gbps cho HDD/SSD
- Thunderbolt: Kết hợp PCIe và DisplayPort, tốc độ cực cao (40 Gbps)

## PHẦN II: HỆ ĐIỀU HÀNH

---

### 2.1. Giới thiệu hệ điều hành

Hệ điều hành (Operating System – OS) là phần mềm hệ thống quan trọng nhất, đóng vai trò trung gian giữa người dùng/phần mềm ứng dụng và phần cứng máy tính.

#### Chức năng chính của hệ điều hành:

1. Quản lý tài nguyên phần cứng: CPU, bộ nhớ, thiết bị lưu trữ, thiết bị I/O
2. Cung cấp giao diện: Command-line (CLI) hoặc đồ họa (GUI) cho người dùng
3. Quản lý tiến trình: Tạo, hủy, lập lịch và đồng bộ các tiến trình
4. Quản lý bộ nhớ: Phân phối, thu hồi và bảo vệ bộ nhớ
5. Quản lý tập tin: Tổ chức, lưu trữ và truy cập dữ liệu
6. Bảo mật: Xác thực, phân quyền và bảo vệ hệ thống
7. Quản lý lỗi: Xử lý lỗi phần cứng và phần mềm

#### Các loại hệ điều hành phổ biến:

- Desktop/Server: Windows, macOS, Linux (Ubuntu, CentOS, Debian, Fedora, v.v.)
- Mobile: Android, iOS
- Real-time OS (RTOS): VxWorks, FreeRTOS, QNX (cho hệ thống nhúng, yêu cầu thời gian thực)
- Embedded OS: Cho thiết bị IoT, router, smart TV

### 2.2. Kernel – Nhân hệ điều hành

Kernel là phần cốt lõi của hệ điều hành, có quyền truy cập trực tiếp vào tất cả phần cứng và tài nguyên hệ thống. Kernel là cầu nối giữa phần mềm ứng dụng và phần cứng.

#### Chức năng của kernel:

- Quản lý tiến trình: Lập lịch CPU, tạo/hủy process, chuyển ngữ cảnh
- Quản lý bộ nhớ: Phân phối RAM, virtual memory, page tables
- Quản lý thiết bị: Giao tiếp với phần cứng thông qua device drivers
- Quản lý file system: Đọc/ghi file, quản lý disk space
- Xử lý interrupts: Phản hồi sự kiện từ phần cứng
- Bảo mật: Thực thi access control, phân quyền
- Networking: Xử lý giao thức mạng, sockets

#### Các loại kernel:

- Monolithic Kernel: Tất cả dịch vụ chạy trong kernel space (Linux, Unix) — Ưu: hiệu năng cao; Nhược: khó bảo trì, dễ crash toàn hệ thống
- Microkernel: Chỉ chức năng tối thiểu trong kernel, dịch vụ khác chạy ở user space (Minix, QNX) — Ưu: ổn định, dễ mở rộng; Nhược: overhead giao tiếp
- Hybrid Kernel: Kết hợp cả hai (Windows NT, macOS XNU)

#### Kernel space vs User space:

- Kernel space: Vùng nhớ dành cho kernel, có quyền truy cập toàn bộ phần cứng
- User space: Vùng nhớ cho ứng dụng, bị giới hạn quyền truy cập
- System call: Cơ chế để ứng dụng yêu cầu kernel thực hiện thao tác (read, write, open, fork, v.v.)

## 2.3. Quản lý tiến trình (Process Management)

Tiến trình (process) là một chương trình đang thực thi. Hệ điều hành phải quản lý nhiều tiến trình đồng thời, phối hợp việc sử dụng CPU và các tài nguyên khác.

### Trạng thái của tiến trình:

- New: Tiến trình đang được tạo
- Ready: Chờ được cấp CPU
- Running: Đang thực thi trên CPU
- Waiting/Blocked: Chờ sự kiện I/O hoặc tài nguyên
- Terminated: Đã hoàn thành hoặc bị hủy

### Process Control Block (PCB):

Mỗi tiến trình được hệ điều hành quản lý thông qua PCB, chứa: Process ID (PID), trạng thái, Program counter, CPU registers, thông tin bộ nhớ (page tables), tài nguyên đang sử dụng (file descriptors), thông tin lập lịch (priority).

### Lập lịch CPU (CPU Scheduling):

- First-Come, First-Served (FCFS): Đơn giản nhưng không tối ưu
- Shortest Job First (SJF): Ưu tiên tiến trình ngắn
- Round Robin: Mỗi tiến trình nhận một time slice
- Priority Scheduling: Ưu tiên theo mức độ quan trọng
- Multilevel Queue: Kết hợp nhiều thuật toán

### Threads – Luồng:

- Thread là đơn vị thực thi nhỏ nhất trong một tiến trình
- Nhiều threads trong cùng process chia sẻ bộ nhớ và tài nguyên
- User-level threads: Quản lý bởi thư viện ở user space
- Kernel-level threads: Quản lý trực tiếp bởi kernel
- Multi-threading: Cho phép song song hóa trong cùng một ứng dụng

## 2.4. Quản lý bộ nhớ (Memory Management)

Quản lý bộ nhớ là một trong những chức năng quan trọng nhất của kernel. Nó đảm bảo mỗi tiến trình có đủ bộ nhớ để hoạt động và không can thiệp vào bộ nhớ của tiến trình khác.

### Bộ nhớ vật lý vs Bộ nhớ ảo:

- Physical Memory: RAM thực tế trong máy
- Virtual Memory: Không gian địa chỉ ảo mà mỗi tiến trình “nhìn thấy”
- MMU (Memory Management Unit): Phần cứng dịch địa chỉ ảo thành địa chỉ vật lý

### Phân trang (Paging):

- Bộ nhớ được chia thành các trang (page) có kích thước cố định (thường 4KB)
- Mỗi tiến trình có page table riêng ánh xạ virtual pages physical frames
- Page Fault: Khi truy cập trang chưa có trong RAM, kernel phải load từ disk
- TLB (Translation Lookaside Buffer): Cache cho việc dịch địa chỉ

### Swap và Page Replacement:

- Swap Space: Vùng trên disk dùng như RAM mở rộng
- Thuật toán thay thế trang: FIFO, LRU (Least Recently Used), Optimal
- Thrashing: Tình trạng hệ thống dành nhiều thời gian swapping hơn là thực thi

### Bảo vệ bộ nhớ:

- Read/Write/Execute bits: Kiểm soát quyền truy cập từng trang
- User/Kernel bit: Phân biệt user mode và kernel mode
- ASLR (Address Space Layout Randomization): Kỹ thuật bảo mật chống buffer overflow

## 2.5. Quản lý tập tin (File System)

File system là cách hệ điều hành tổ chức và quản lý dữ liệu trên thiết bị lưu trữ.

### Các khái niệm cơ bản:

- File: Đơn vị lưu trữ dữ liệu logic
- Directory/Folder: Cấu trúc phân cấp để tổ chức files
- Path: Đường dẫn đến file (absolute hoặc relative)
- File Descriptor: Số nguyên đại diện cho file đang mở
- Inode: Cấu trúc dữ liệu chứa metadata của file (permissions, owner, timestamps, pointers đến data blocks)

### Các loại file system phổ biến:

- FAT32/exFAT: Đơn giản, tương thích rộng, không có journaling
- NTFS: File system mặc định của Windows, hỗ trợ permissions, encryption, compression
- ext4: File system phổ biến cho Linux, journaling, hiệu năng tốt
- APFS: File system của macOS/iOS, tối ưu cho SSD
- ZFS: File system tiên tiến, hỗ trợ snapshot, compression, checksum

### File permissions (Unix/Linux):

- chmod: Thay đổi permissions — rwx: Read, Write, Execute
- User/Group/Other: Phân quyền cho owner, group, và others
- Ví dụ: 755 = rwxr-xr-x (owner: full, group: read+execute, others: read+execute)

## 2.6. Quản lý thiết bị (Device Management)

Hệ điều hành phải quản lý nhiều loại thiết bị khác nhau, từ bàn phím, chuột đến card mạng, GPU.

### Device Drivers:

- Driver là phần mềm cho phép kernel giao tiếp với phần cứng cụ thể



- Mỗi loại thiết bị cần driver tương thích
- Kernel modules: Driver có thể được load/unload động
- Plug and Play: Tự động phát hiện và cài driver khi kết nối thiết bị mới

### Interrupts – Ngắt:

- Interrupt là tín hiệu từ thiết bị yêu cầu CPU chú ý
- Hardware Interrupts: Từ thiết bị ngoại vi (keyboard, disk, network)
- Software Interrupts: System calls, exceptions
- Interrupt Handler: Code trong kernel xử lý interrupt

### DMA – Direct Memory Access:

Cho phép thiết bị I/O truy cập RAM trực tiếp mà không cần CPU — giảm tải cho CPU, tăng hiệu năng.

## 2.7. Hệ điều hành phổ biến

Linux: Open-source, miễn phí, tùy biến cao. Monolithic kernel với module load động. Phổ biến: Server, supercomputer, embedded, cloud. Ưu: bảo mật, hiệu năng, cộng đồng lớn. Nhược: learning curve, driver support.

Windows: Commercial, proprietary. Hybrid kernel (Windows NT). Phổ biến: Desktop, enterprise, gaming. Ưu: user-friendly, driver support, software compatibility. Nhược: chi phí, bảo mật, resource-heavy.

macOS: Commercial, chỉ chạy trên phần cứng Apple. Hybrid kernel (XNU). Ưu: UX tốt, tích hợp phần cứng-phần mềm, bảo mật. Nhược: chỉ chạy trên Mac, chi phí cao.

Android: Open-source (dựa trên Linux kernel). Phổ biến: Mobile devices. Ưu: miễn phí, tùy biến, app ecosystem lớn. Nhược: fragmentation, security concerns.

## PHẦN III: MẠNG MÁY TÍNH

### 3.1. Giới thiệu mạng máy tính

Mạng máy tính là tập hợp các máy tính và thiết bị được kết nối với nhau để chia sẻ tài nguyên, dữ liệu và dịch vụ.

#### Lợi ích của mạng máy tính:

- Chia sẻ tài nguyên (file, printer, internet connection)
- Giao tiếp (email, messaging, video conferencing)
- Truy cập dữ liệu từ xa
- Redundancy và fault tolerance
- Scalability

#### Phân loại mạng theo khoảng cách:

- PAN (Personal Area Network): Vài mét (Bluetooth, USB)
- LAN (Local Area Network): Tòa nhà, khuôn viên (Ethernet, WiFi)
- MAN (Metropolitan Area Network): Thành phố
- WAN (Wide Area Network): Quốc gia, toàn cầu (Internet)

#### Topology mạng:

- Bus: Tất cả thiết bị kết nối vào một đường truyền chung
- Star: Tất cả thiết bị kết nối vào trung tâm (switch/hub)
- Ring: Mỗi thiết bị kết nối với hai thiết bị lân cận
- Mesh: Mỗi thiết bị kết nối với nhiều thiết bị khác
- Tree / Hybrid: Kết hợp nhiều topology

### 3.2. Mô hình OSI và TCP/IP

Mô hình phân tầng giúp chuẩn hóa và đơn giản hóa việc thiết kế mạng.

#### Mô hình OSI (7 layers):

| Layer | Tên          | Chức năng                             | Protocol / Device ví dụ    |
|-------|--------------|---------------------------------------|----------------------------|
| 7     | Application  | Giao diện ứng dụng-mạng               | HTTP, FTP, SMTP, DNS       |
| 6     | Presentation | Dịch dữ liệu, encryption, compression | SSL/TLS, JPEG, MPEG        |
| 5     | Session      | Quản lý session giữa applications     | NetBIOS, RPC               |
| 4     | Transport    | Truyền đầu-cuối, reliability          | TCP, UDP                   |
| 3     | Network      | Định tuyến, logical addressing        | IP, ICMP, Router           |
| 2     | Data Link    | Physical addressing, error detection  | Ethernet, MAC, Switch      |
| 1     | Physical     | Truyền bit qua môi trường vật lý      | Cable, Hub, Repeater, WiFi |

## Mô hình TCP/IP (4 layers):

| Layer | Tên            | Protocol                  |
|-------|----------------|---------------------------|
| 4     | Application    | HTTP, FTP, SMTP, DNS, SSH |
| 3     | Transport      | TCP, UDP                  |
| 2     | Internet       | IP, ICMP, ARP             |
| 1     | Network Access | Ethernet, WiFi, PPP       |

Dữ liệu đi xuống từ Application Network Access (encapsulation), mỗi layer thêm header riêng. Khi nhận, dữ liệu đi lên (decapsulation), mỗi layer loại bỏ header tương ứng.

## 3.3. Các giao thức mạng cơ bản

### TCP (Transmission Control Protocol):

- Connection-oriented: Thiết lập kết nối trước khi truyền dữ liệu (3-way handshake)
- Reliable: Đảm bảo dữ liệu đến đúng thứ tự, không mất, không lỗi
- Flow control & Congestion control
- Use cases: Web (HTTP/HTTPS), email (SMTP), file transfer (FTP)

### UDP (User Datagram Protocol):

- Connectionless, Unreliable, Fast (overhead thấp, latency thấp)
- Use cases: Streaming, VoIP, online gaming, DNS queries

### IP (Internet Protocol):

- Routing: Chuyển packet từ source đến destination
- IPv4: 32-bit address (192.168.1.1) — IPv6: 128-bit address
- Best-effort delivery: Không đảm bảo packet đến nơi

### HTTP/HTTPS & DNS:

- HTTP: Giao thức ứng dụng cho web; HTTPS: HTTP qua SSL/TLS (encrypted)
- Methods: GET, POST, PUT, DELETE, PATCH — Status codes: 200, 404, 500, ...
- DNS: Phân giải tên miền (example.com IP). Record types: A, AAAA, CNAME, MX, TXT

## 3.4. Địa chỉ IP và định tuyến

### IPv4 Addressing:

- Format: 4 octets, mỗi octet 0–255 (32 bits total)
- Private IP ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
- Subnet Mask: Xác định phần network và phần host — CIDR: 192.168.1.0/24

### Routing & DHCP:

- Router: Thiết bị chuyển packet giữa các mạng; Routing table; Default gateway
- Dynamic routing protocols: RIP, OSPF, BGP

- NAT: Chuyển đổi giữa private và public IP
- DHCP: Tự động cấp phát địa chỉ IP — DORA: Discover Offer Request Acknowledge

### 3.5. Các thiết bị mạng

NIC (Network Interface Card): Card mạng; MAC Address (48-bit); Ethernet (RJ45) / WiFi

Hub: Layer 1, broadcast tất cả traffic — ít được dùng hiện nay

Switch: Layer 2, học MAC address và forward traffic chỉ đến port đích

Router: Layer 3, kết nối nhiều mạng, định tuyến dựa trên IP address

Firewall: Kiểm soát traffic vào/ra — Packet filtering, Stateful inspection

Access Point (AP): Cho phép kết nối WiFi; Standards: 802.11a/b/g/n/ac/ax (WiFi 6)

### 3.6. Bảo mật mạng cơ bản

#### Các mối đe dọa phổ biến:

- Malware: Virus, trojan, ransomware, spyware
- Phishing, DDoS, Man-in-the-Middle (MitM)
- SQL Injection, XSS (Cross-Site Scripting)

#### Biện pháp bảo vệ:

- Firewall, Encryption (SSL/TLS, VPN, SSH)
- Authentication (Password, 2FA, biometrics), Access Control (RBAC, least privilege)
- Regular updates, Network segmentation, IDS/IPS

#### SSL/TLS & VPN:

- SSL/TLS: Encryption + Handshake + Certificate HTTPS
- VPN: Tunnel encrypted qua internet — Remote access / Site-to-site — Protocols: IPsec, OpenVPN, WireGuard

## PHẦN IV: CÁCH MÁY TÍNH THỰC SỰ HOẠT ĐỘNG

---

### 4.1. Từ code đến thực thi

Hiểu cách code được biến thành chương trình thực thi giúp bạn viết code tốt hơn và debug hiệu quả hơn.

#### Quá trình compile (C/C++ ví dụ):

1. Preprocessing: Xử lý directives (#include, #define, macros)
2. Compilation: Chuyển source code assembly code
3. Assembly: Chuyển assembly machine code (object file)
4. Linking: Kết hợp nhiều object files + libraries executable

#### Executable format (ELF trên Linux):

Header, Program Header Table, Sections (.text, .data, .bss, .rodata), Symbol Table, Dynamic Linking (.so files).

#### Quá trình thực thi:

1. OS tạo process mới, allocate memory
2. Loader đọc executable, map vào virtual address space
3. Dynamic linker load shared libraries
4. CPU bắt đầu thực thi từ entry point (\_start main())
5. Program chạy, thực hiện system calls khi cần
6. Program kết thúc, OS reclaim resources

#### Interpreter (Python, JavaScript):

Source code Bytecode Interpreter executes. JIT (Just-In-Time) compile bytecode machine code khi chạy (V8, PyPy, JVM). Trade-off: linh hoạt hơn, nhưng thường chậm hơn compiled language.

### 4.2. Chu trình boot và khởi động

Quá trình boot là chuỗi các bước từ khi bật máy đến khi hệ điều hành sẵn sàng.

#### Quy trình boot truyền thống (BIOS):

1. Power On POST (Power-On Self Test)
2. BIOS Initialization Boot Device Selection
3. MBR (Master Boot Record) Bootloader (GRUB, LILO)
4. Kernel Load Kernel Initialization
5. Init Process (PID 1: systemd/init) Login/Display Manager

#### UEFI:

Thay thế BIOS, hỗ trợ disk lớn hơn, boot nhanh hơn. Secure Boot, FAST Boot, EFI System Partition (ESP).

---

## 4.3. Hệ thống nhúng và hệ điều hành

Hệ thống nhúng (embedded systems) là máy tính chuyên dụng cho một mục đích cụ thể, thường có resource hạn chế.

Đặc điểm: Resource-constrained, Real-time requirements, Low power, Reliability, Custom hardware.

RTOS: Hard real-time (FreeRTOS, VxWorks) vs Soft real-time. Preemptive scheduling, Priority inversion.

MCU vs MPU: Microcontroller = CPU + RAM + ROM + I/O trên một chip (AVR, ARM Cortex-M, ESP32).  
Microprocessor = chỉ CPU, cần external RAM/storage (ARM Cortex-A, x86).

Embedded Linux: Yocto/Buildroot, Device Tree, Cross-compilation, Minimal root filesystem.

## 4.4. Tối ưu hóa và hiệu năng

### CPU optimization:

- Cache-friendly code: spatial & temporal locality, tránh cache thrashing
- Branch prediction, Instruction-level parallelism, Vectorization (SIMD)
- Compiler optimizations (-O2, -O3), loop unrolling

### Memory / I/O / Network optimization:

- Reduce allocations, data locality, tránh fragmentation, minimize page faults
- Buffering, Async I/O (epoll, io\_uring), mmap(), Direct I/O
- Connection pooling, Batching, Compression, CDN, HTTP/2-3 / QUIC

### Profiling tools:

CPU: perf, gprof, VTune • Memory: Valgrind, AddressSanitizer • I/O: iostat, iotop • Network: Wireshark, tcpdump • System: top, htop, vmstat

## PHẦN V: TỔNG KẾT VÀ ỨNG DỤNG

---

### 5.1. Tích hợp các thành phần

Khi bạn hiểu phần cứng, hệ điều hành và mạng, bạn có thể nhìn thấy bức tranh toàn cảnh về cách máy tính hoạt động.

**Ví dụ: Khi bạn gõ một URL vào browser:**

1. Keyboard input: Hardware interrupt kernel userspace browser
2. DNS lookup: Application system call network stack DNS server resolve domain to IP
3. TCP connection: 3-way handshake (SYN, SYN-ACK, ACK)
4. HTTP request Server processing Response
5. Rendering: Browser parses HTML/CSS, executes JS, renders page using GPU Display

**Ví dụ: Khi bạn lưu file:**

1. Application: Calls write() system call
2. Kernel: Copies data to page cache
3. File system: Updates metadata (inode, directory entry)
4. Block layer Device driver Disk writes data to sectors
5. Journal: Logs changes for recovery (nếu có journaling)

### 5.2. Best practices cho kỹ sư

**Khi viết code:**

- Hiểu complexity (time và space) của thuật toán
- Optimize cho common case; Profile trước khi optimize (premature optimization là kẻ thù)
- Write readable code, sau đó optimize bottlenecks
- Understand memory model của ngôn ngữ (stack vs heap, GC, v.v.)

**Khi debug:**

- Reproduce bug consistently; Use logging và debugging tools (gdb, lldb, strace)
- Check system logs; Understand error messages (kernel panic, segfault, OOM killer)
- Use monitoring tools để identify patterns

**Khi thiết kế hệ thống:**

- Consider scalability (horizontal vs vertical)
- Understand trade-offs (consistency vs availability, latency vs throughput)
- Design for failure (redundancy, failover, retries)
- Security by design (least privilege, defense in depth)
- Monitor và alert (metrics, logs, traces)

**Continuous learning:**

- Đọc source code (Linux kernel, open-source projects)

- Thực hành với tools (gdb, perf, Wireshark, strace)
- Follow blogs, papers (Brendan Gregg, ACM Queue, USENIX)
- Build projects (OS, compiler, network stack từ scratch)

## 5.3. Tài nguyên học tập

### Sách:

- Computer Systems: A Programmer's Perspective (CS:APP) – Randal Bryant, David O'Hallaron
- Operating System Concepts – Abraham Silberschatz
- Computer Networking: A Top-Down Approach – Kurose & Ross
- The Linux Programming Interface – Michael Kerrisk
- Designing Data-Intensive Applications – Martin Kleppmann
- Systems Performance – Brendan Gregg

### Online resources:

- OSDev.org – Operating system development community
- Brendan Gregg's blog – Performance engineering
- Linux Kernel Documentation: docs.kernel.org
- Cloudflare Blog, High Scalability, Coursera/edX

### Projects để thực hành:

- Build a simple OS (OSDev tutorials)
- Write a compiler; Implement network stack; Build a database (SQLite clone)
- Contribute to open-source; Reverse engineer binaries

### Tools để master:

Debugging: gdb, lldb, strace, ltrace • Profiling: perf, VTune, Valgrind • Network: Wireshark, tcpdump, ss  
• System: top, htop, vmstat, iostat • Containers: Docker, Kubernetes, QEMU, KVM



---

## LỜI KẾT

---

Kiến thức về phần cứng, hệ điều hành và mạng là nền tảng vững chắc cho bất kỳ kỹ sư phần mềm nào. Nó giúp bạn:

- Viết code hiệu quả hơn, tận dụng được sức mạnh phần cứng
- Debug và troubleshoot nhanh hơn khi hiểu điều gì đang xảy ra “bên dưới”
- Thiết kế hệ thống tốt hơn, đưa ra quyết định kiến trúc đúng đắn
- Học công nghệ mới nhanh hơn, vì nhiều công nghệ mới chỉ là biến thể của nguyên lý cơ bản
- Giao tiếp tốt hơn với các kỹ sư phần cứng, DevOps, SRE

Đường đi để master những kiến thức này không ngắn, nhưng mỗi bước đi đều đáng giá. Bắt đầu từ những nguyên lý cơ bản, thực hành thường xuyên, và không ngừng học hỏi. Chúc bạn thành công trên hành trình trở thành kỹ sư phần mềm xuất sắc!

---

— Hết —

Biên soạn bởi DigitalCore Team — Phiên bản V1.0

© 2026 DigitalCore Team